

The National Higher School of Artificial Intelligence
Introduction to AI Course Project
Spring 2026

Introduction to AI Project Paper

Project Title: Emergency Ambulance Dispatch Routing (Project 7)

Student list:

Full name	Group
Allali Bouchra	G1
Bentata Ikram Amina	G8
Ferhaoui Amina	G12
Hamadach Thawriyya	G12
Immessaoudene Malak Ikram	G1
Moumani Malek Nourhene	G8

Abstract.....	1
Introduction.....	1
State of the Art.....	2
Data Set Building.....	3
Problem Solving techniques.....	4
Application Design	6
Results and Analysis	6
Discussion.....	10
Conclusion	10
References.....	11
Appendix A	
Appendix B	

Abstract

Emergency response systems depend heavily on fast ambulance dispatch and efficient routing, especially in urban environments where traffic congestion and uneven emergency distribution can significantly increase response delays. This project presents a simulation-based intelligent ambulance dispatch and routing system designed to improve emergency response performance using artificial intelligence techniques and graph-based optimisation methods.

The system models the road network as a weighted directed graph and combines traffic-aware A* routing with dynamic ambulance repositioning based on Hill Climbing optimisation. Traffic conditions are simulated using dynamic edge-weight multipliers, while emergency events are generated stochastically using a Poisson process. The objective of the system is to reduce ambulance response time and improve emergency coverage under varying traffic and demand conditions.

Several simulation experiments were conducted to compare dynamic dispatch and repositioning strategies against static and distance-based approaches. The obtained results showed that traffic-aware routing and dynamic ambulance positioning generally improved response efficiency and reduced delays during surge scenarios involving multiple simultaneous emergencies.

Although the project was developed within a simulation framework, the experiments demonstrate how search algorithms, optimisation methods, and dynamic routing techniques can be combined to support intelligent emergency dispatch systems. The proposed approach also provides a flexible environment for evaluating ambulance allocation strategies under different urban conditions.

Keywords: Ambulance Dispatch, A* Search, Hill Climbing, Traffic-Aware Routing, Emergency Response Simulation

Introduction

Emergency medical response is one of the most time-sensitive services in modern urban systems. In many emergency situations, response delays can directly affect patient outcomes, especially during critical incidents such as cardiac arrest, severe trauma, or road accidents. Because of increasing urbanisation and traffic congestion, efficiently dispatching ambulances has become a complex operational problem that requires both fast routing and effective resource allocation.

Traditional ambulance dispatch systems often rely on static stationing strategies or simple distance-based assignment methods. While these approaches are relatively easy to implement, they do not always perform well in dynamic urban environments where traffic conditions and emergency demand change continuously throughout the day. The geographically closest ambulance is not necessarily the fastest option once congestion and road-network constraints are taken into account.

Artificial intelligence techniques and graph-based routing algorithms provide alternative approaches for improving emergency response performance. Search algorithms such as A* are widely used in pathfinding problems because they can efficiently compute near-optimal routes on weighted graphs while considering movement cost and heuristic estimates (Russell & Norvig, 2021). Similarly, optimisation methods such as Hill Climbing can be used to improve ambulance positioning by relocating idle vehicles toward strategically useful areas of the city.

The objective of this project is to design and evaluate a simulation-based ambulance dispatch and routing system capable of adapting to changing traffic conditions and emergency demand. The system combines traffic-aware routing using the A* algorithm with dynamic ambulance repositioning using Hill Climbing optimisation. Emergency events are generated dynamically during the simulation, and road congestion is represented through changing edge weights on the road graph.

The project focuses mainly on analysing the impact of dynamic routing and repositioning strategies on emergency response performance. Different dispatch approaches are compared under multiple traffic conditions and surge scenarios in order to evaluate routing efficiency, ambulance coverage, and response stability.

Unlike real-world emergency response systems, the proposed solution is intended primarily for simulation and experimentation purposes. Nevertheless, the project demonstrates how AI search and optimisation techniques can be integrated into intelligent dispatch systems and evaluated within a realistic graph-based environment

State of the Art

The problem of emergency vehicle routing and ambulance dispatch has been widely studied in operations research, transportation systems and artificial intelligence. Most of the existing research is mainly focused on reducing the emergency response time, improving ambulance allocation and adapting dispatch decisions to the dynamic urban conditions.

Graph-based pathfinding is one of the most widely used techniques in routing systems. Hart Nilsson and Raphael introduced the A* algorithm (Hart, Nilsson, & Raphael, 1968). A* efficiently computes routes by combining the actual cost of the path with a heuristic estimate, minimizing unnecessary exploration. Since then, the algorithm has been used in navigation systems, robotics, logistics and emergency routing.

The problem of ambulance location and relocation has been studied by a number of authors using optimisation methods. Brotcorne Laporte and Semet studied ambulance location and relocation models in order to improve emergency coverage and reduce response delays. (Brotcorne, Laporte, & Semet, 2003) They showed that it is important to dynamically locate ambulances according to expected emergency demand rather than relying on static standby locations.

Of great importance in emergency routing problems are also dynamic traffic conditions. Ichoua Gendreau and Potvin have studied real time dispatching on traffic conditions, it demonstrated that static stationing are not as efficient especially regarding time evolution on traffic and roads (Ichoua, Gendreau, & Potvin, 2000). This disadvantage is especially important for the urban emergency systems where road conditions may change very fast in the heavy traffic periods.

Many existing emergency response simulations and dispatching systems also focus mainly on shortest-distance routing without fully considering dynamic repositioning strategies. On the contrary, the method proposed in this project integrates traffic aware routing and dynamic repositioning of ambulances in a single simulation environment. The aim is not to build a fully-fledged real-world EMS platform but to assess the impact of AI search and optimisation techniques on emergency response performance in a changing urban environment.

Therefore, in contrast to simpler dispatch simulations, the proposed system combines graph-based pathfinding, dynamic traffic modeling, stochastic emergency generation and ambulance repositioning in a single experimental environment.

Data Set Building

To be noted: The dataset we worked on depends entirely on real depots, hospital, map configs and real Road speed data. The only “Speculated” detail is the hospital capacity since, when gathering the necessary info and intel, was deemed as “Private” and cannot be shared with us for a simple project with no official authorization.

The simulation environment used in this project was built around a graph-based representation. The dataset was designed specifically to support ambulance dispatching, routing, and repositioning experiments under different operational scenarios.

The road network was represented as a weighted directed graph where nodes correspond to intersections or important locations, while edges represent roads connecting these points.

Table 1.Road Network Characteristics

Feature	Value
Number of Nodes	53,616
Number of Edges	130,871
Graph Type	Directed Weighted Graph
Stored Attributes	Node Coordinates, Edge Weights
Road Categories	Motorway, Primary, Secondary, Residential

The graph data was stored using JSON-based structures in order to simplify loading and reuse across the different modules of the system. During preprocessing, invalid connections and isolated nodes were removed to ensure that routing algorithms could compute valid paths between ambulance depots, hospitals, and emergency locations.

Additional preprocessing was applied to road weights in order to simulate changing traffic conditions. Different traffic periods were introduced by modifying edge weights according to road category and congestion level. This allowed the routing system to adapt dynamically during the simulation.

Table 2.Traffic Simulation Parameters

Traffic Period	Motorway Multiplier	Primary Road Multiplier
Peak Traffic	3.0x	2.0x
Normal Traffic	1.5x	1.2x
Night Traffic	0.5x	0.5x

To make the environment less predictable, random congestion events were also introduced during simulation runs. These events temporarily increased the cost of selected roads and forced the dispatch system to recompute routes dynamically according to current traffic conditions.

Emergency events were generated synthetically using a Poisson distribution. This approach allowed emergencies to appear continuously during the simulation while preserving randomness between different runs. Each generated emergency contains a timestamp, a location on the graph, and a severity level used by the dispatch system.

Table 3.Emergency Generation Parameters

Parameter	Value
Base Emergency Rate	0.8 emergencies/hour
Critical Emergencies	10%
High Severity	25%
Medium Severity	45%
Low Severity	20%

Peak Hour Multiplier	2.5x
----------------------	------

Emergency distribution was not uniform across the environment. Urban areas generated emergencies more frequently than suburban or industrial regions, creating high-demand zones that influenced ambulance positioning and dispatch decisions throughout the experiments.

The dataset also included hospitals and ambulance depots placed on valid graph nodes. Ambulances moved between emergency locations and hospitals while remaining constrained by the road network and current traffic conditions. The placement of depots and hospitals across the graph helped provide broader coverage during simulation runs.

Several validation procedures were performed before running the experiments. Connectivity checks ensured that major regions of the graph remained reachable, while coordinate verification helped confirm that all generated locations stayed within the expected simulation area. Additional tests were performed to verify that routing algorithms could successfully compute paths between hospitals, depots, and emergency locations without failure.

Several visualisations were generated from the dataset in order to analyse the simulation environment more effectively. These included road-network plots, traffic heatmaps, emergency distribution maps, ambulance movement visualisations, and response-time histograms. The generated figures helped identify congestion zones, emergency hotspots, and routing behaviour during different simulation scenarios.

Overall, the constructed dataset provided a flexible environment for evaluating ambulance dispatch and routing strategies under dynamic urban conditions. The combination of graph-based routing, traffic variation, and stochastic emergency generation allowed the implemented AI techniques to be tested under a wide range of simulation scenarios.

Problem Solving techniques

The Emergency Ambulance Dispatch and Routing system was designed as a dynamic optimization and pathfinding problem in which ambulances must react to continuously arriving emergency calls while considering traffic conditions, ambulance availability, and hospital accessibility. The objective of the system is to minimize emergency response time while maintaining efficient ambulance coverage throughout the city.

Problem Formulation

The road network is modeled as a weighted graph where intersections are represented as nodes and roads are represented as edges. Each edge contains a weight corresponding to travel time. Since traffic conditions may change dynamically, edge weights can increase or decrease during simulation execution.

When an emergency occurs, the system must perform three main tasks:

Select the best available ambulance.

Compute the fastest route from the ambulance to the emergency location.

Compute the fastest route from the emergency location to the nearest hospital.

The main optimization objective is minimizing the total response time for all emergencies:

$$\min \sum_{i=1}^n \text{ResponseTime}_i$$

where n represents the total number of emergencies handled during the simulation. The response time includes dispatch delay and ambulance travel time.

State Representation

The simulation environment contains several dynamic entities including ambulances, hospitals, emergencies, and traffic conditions.

The road network is represented using:

- Nodes: intersections, hospitals, depots
- Edges: roads connecting nodes
- Edge weights: travel time affected by traffic

Each ambulance is represented by:

- Ambulance ID
- Current position
- Availability status
- Assigned emergency
- Current route

The ambulance state machine contains the following states:

- Idle
- Dispatched
- At scene
- To hospital

State transitions occur dynamically during simulation execution.

Greedy Dispatch Algorithm

The first dispatch strategy implemented in the project is a Greedy dispatch algorithm. When an emergency occurs, the system selects the geographically closest available ambulance using Euclidean distance.

The Euclidean distance is calculated using:

$$d(a, b) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The Greedy dispatch process operates as follows:

Detect new emergency.

Retrieve all available ambulances.

Compute Euclidean distance between each ambulance and the emergency.

Select the ambulance with minimum distance.

Dispatch the selected ambulance.

The main advantage of this method is its simplicity and low computational cost. However, the algorithm ignores traffic congestion and dynamic road conditions. As a result, the geographically closest ambulance may not always provide the shortest travel time.

A* Search

To improve routing quality, the project uses Real-Time A* search for pathfinding. A* was selected because it efficiently computes near-optimal routes while exploring fewer nodes than uninformed search algorithms.

The A* evaluation function is:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the travel cost from the start node to node n
- $h(n)$ is the heuristic estimate from node n to the destination

The heuristic function used in this project is the Euclidean distance divided by the maximum road speed. This heuristic is admissible because it never overestimates the real travel cost.

A* dynamically recalculates routes whenever traffic conditions change. If congestion appears on a road segment, edge weights are updated and a new optimal route is generated. The advantages of A* include:

- Reduced response time
- Dynamic adaptation to traffic
- Efficient route optimization
- Reduced node exploration

The algorithm is used both for:

- Ambulance-to-emergency routing
- Emergency-to-hospital routing

Hill Climbing for Ambulance Repositioning

The project also addresses ambulance standby positioning. When ambulances become idle after delivering a patient to a hospital, they are repositioned to strategic standby locations to maximize city coverage.

This optimization problem is solved using Hill Climbing.

A Hill Climbing state is represented as the set of standby positions for all idle ambulances. The fitness function evaluates the average response time to historical emergency locations:

$$Fitness(S) = \frac{1}{m} \sum_{i=1}^m ResponseTime_i$$

where m is the number of historical emergencies.

The Hill Climbing algorithm starts with random ambulance positions and iteratively searches neighboring configurations. If a neighboring state produces lower average response time, the system moves to that state. To avoid local optima, random restarts were added to the algorithm. Multiple runs are executed with different initial positions and the best final configuration is selected.

Comparative Evaluation

Several experiments were conducted to evaluate the effectiveness of the implemented techniques.

Greedy Dispatch vs A*-Based Dispatch

The Greedy dispatch strategy was compared with A*-based dispatch using:

- Average response time
- Maximum response time
- Total travel distance

The experiments showed that A*-based dispatch performs significantly better under congested traffic conditions because it considers real travel cost instead of straight-line distance.

Application Design

The application is realised as a self-contained single-process Python desktop environment in which geographic visualisation, traffic simulation, and concurrent routing computation coexist without inter-process communication overhead. Architectural coherence is maintained through a rigorous separation of concerns: a stateful backend layer governs the entirety of simulation logic, whilst a rendering layer — implemented atop PySide6 and PyQtGraph — remains exclusively responsible for visual output and operator interaction. The two layers are decoupled by a dual-timer pipeline comprising an independent 30 Hz simulation tick and a 60 Hz render tick, ensuring that computationally intensive routing operations cannot induce frame-rate degradation in the graphical interface.

Module Structure

The system is partitioned into six discrete modules, each encapsulating a single architectural concern. Their responsibilities are enumerated below:

Table 4. Application Module Responsibilities

Module	Responsibility
traffic_model.py	Maintains road-class congestion multipliers across three operational periods and constructs the full-city heatmap intensity grid via Gaussian hotspot splatting and per-edge OSM weight rasterisation.
sim_adapter.py	Central simulation controller: owns ambulance and emergency state collections, the Poisson spawner, a KD-tree spatial index for node snapping, and the ThreadPoolExecutor managing twelve asynchronous routing workers.
view_models.py	Dependency-free dataclasses (AmbView, EmView, PulseRing, RouteCache) shared safely across threads without incurring Qt or SciPy import cost.
heatmap.py	Pre-computes per-edge base congestion scores from a 14-level OSM highway-weight table; on each rebuild, scales scores by a temporal multiplier and consolidates co-styled segments into single PlotDataItem draw calls.
map_animation.py	PyQtGraph rendering layer: constructs all plot items, hosts the DispatchWindow QMainWindow, and mediates both Qt timers.

colors.py	Single source of truth for the visual palette. Defines THEME_DAY, THEME_RUSH, and THEME_NIGHT constants; exposes apply_theme() and road_style_for_theme(), which mutate the global colour dictionary in place so that every rendering component inherits updated values without explicit propagation.
-----------	---

Rendering Pipeline and Road Geometry

Road geometry is rendered through a batching strategy specifically designed to accommodate the scale of the Algiers graph. All 130,871 directed edges are grouped at startup into six style buckets according to their OSM highway classification. Within each bucket, segment coordinates are concatenated into a single NaN-delimited floating-point array and submitted as one PlotDataItem, reducing the Qt scene-graph from a theoretically unbounded number of individual draw calls to precisely six — a reduction that sustains interactive frame rates at full graph scale. When the active theme changes, road_style_for_theme() returns revised (colour, line-width, opacity) tuples for each bucket and the six items are re-styled in place; no geometric data is reconstructed.

Traffic Heatmap Overlay

The heatmap constitutes a full-extent translucent image overlay spanning the complete geographic bounding box of the simulation, composited above the road layer and below all dynamic entities (Z-order values 2, 5, and 11 respectively). It is produced by blending two independent intensity sources. The static hotspot layer is constructed by the TrafficModel at initialisation: 37 empirically grounded Algiers congestion anchors — derived from OpenStreetMap classifications and community traffic intelligence — are Gaussian-splatted onto a 120×120 float32 grid, after which a rasterised per-edge OSM intensity contribution is superimposed via seed_graph_heatmap(). The composite is smoothed with scipy.ndimage.gaussian_filter($\sigma = 4$) to eliminate discontinuities. The live simulation layer accumulates emergency call coordinates on a secondary 60×60 counter grid throughout the run; this grid is upsampled and blended at a 35 % weight so that genuine demand hotspots emerging during a session progressively reinforce the background distribution. The resulting array is mapped through a time-sensitive RGBA ramp — warm orange-red under normal conditions, deep red during rush hour, and desaturated blue at night — and the ImageItem is refreshed asynchronously every 90 render frames, approximately every three seconds, to avoid burdening the render tick with array operations.

Theming and Adaptive Night Mode

Depressing the T key advances the simulation through three operationally distinct time periods. Each transition atomically invokes apply_theme() in colors.py, overwriting the global C dictionary with the corresponding palette variant. Night mode additionally re-applies the Qt stylesheet across the entire widget hierarchy — recolouring panel backgrounds, scrollbar handles, tooltip borders, and the PyQtGraph plot background in a single pass. Because all custom widgets read directly from C within their paintEvent handlers, no explicit propagation is required; the subsequent render frame reflects the updated palette uniformly.

Interface Panels and Auxiliary Overlays

The right-hand panel hosts five purpose-built widgets, each refreshed in batches every four render frames to suppress redundant repaints: a TimeBadge displaying the active time period; a ModeBadge presenting per-mode average response times alongside the latest hill-climb optimisation cost; a QueueBanner reporting queue depth and maximum waiting time; a scrollable UnitRoster enumerating all ambulances with colour-coded state indicators and last-recorded response times; and a colour-stratified IncidentLog carrying timestamped event entries. The header row exposes five StatChip indicators covering total dispatches, resolution rate, mean and ninety-fifth-percentile response times, fleet utilisation, and instantaneous queue depth.

Three auxiliary overlays augment the base map. Voronoi service regions are rendered by assigning a stratified sample of up to 1,200 graph nodes to their nearest ambulance and computing the convex hull of each resulting cluster via scipy.spatial.ConvexHull, producing semi-transparent filled polygons that depict effective unit coverage. Hill-climb standby recommendations are superimposed as dashed amber rings scaled proportionally to the active zoom level. Newly spawned emergencies additionally generate expanding pulse rings that decay in opacity over successive frames, providing a transient visual cue for the operator.

Technologies

Table 5. Backend Technologies

Technology	Purpose	Version
Python	Core runtime for all simulation, routing, and optimisation modules	3.11
NumPy	Batched coordinate arrays, RGBA image construction, statistical aggregation	1.26
SciPy	KDTree node snapping, ConvexHull for Voronoi regions, gaussian_filter for heatmap blending	1.13
ThreadPoolExecutor	12-worker asynchronous pool isolating routing and hill-climb tasks from the simulation tick	stdlib

Table 6. Frontend Technologies

Technology	Purpose	Version
PySide6	Qt 6 Python bindings: main window, layouts, stylesheets, keyboard event handling	6.7
PyQtGraph	High-throughput PlotDataItem road batches, ScatterPlotItem markers, ImageItem heatmap overlay	0.13
Qt Graphics Framework	QGraphicsPixmapItem ambulance/emergency sprites, QGraphicsEllipseItem pulse rings and standby rings	—
Qt Dual-Timer Pipeline	Independent QTimer instances enforcing the 30 Hz simulation / 60 Hz render decoupling	—

Results and Analysis

Simulation experiments were designed to evaluate the proposed system via comparison of different dispatching and ambulance positioning strategies for different traffic and demand conditions. The evaluation mainly focused on the response time, emergency coverage, queue behavior during surge situations and routing efficiency under different traffic conditions. To ensure fair comparison between methods, identical emergency sequences were reused across all experiments.

The first experiment compared the Greedy Euclidean dispatch strategy with the A*-based dispatch strategy. The Greedy method selects the geographically closest available ambulance without considering traffic conditions or actual travel cost on the road network. In contrast, the A*-based strategy selects ambulances according to estimated travel time using dynamic edge weights. The results showed that the A*-based approach generally obtained lower response time and more stable performance in different traffic environments. Table 1 presents the overall comparison between both dispatch strategies.

Method	Average Response Time (min)	Maximum Response Time (min)	Emergency Coverage
Greedy Dispatch	8.47	14.2	87.3%
A*-Based Dispatch	6.82	11.1	92.7%

Table 7. Dispatch Strategy Comparison

The obtained results indicate that considering congestion and actual travel cost during dispatch decisions improves ambulance routing efficiency. In many simulations runs, the geographically closest ambulance was not necessarily the fastest option due to congestion on major roads. The A*-based approach was able to

reduce average travel delays and improve emergency coverage when traffic conditions were taken into account.

The system performance was significantly affected by traffic conditions as well. During rush-hour periods, congestion increased the travel cost of several road segments, making traffic-aware routing more beneficial. Under lighter traffic conditions, the difference between both strategies became smaller but still remained visible.

<i>Traffic Condition</i>	<i>Greedy Dispatch (min)</i>	<i>A*-Based Dispatch (min)</i>
<i>Peak Traffic</i>	<i>11.23</i>	<i>8.91</i>
<i>Normal Traffic</i>	<i>8.47</i>	<i>6.82</i>
<i>Night Traffic</i>	<i>5.12</i>	<i>4.28</i>

Table 8. Performance Under Different Traffic Conditions

The largest difference between the two dispatch methods was observed during peak congestion periods. In these situations, the shortest geometric path often produced longer travel times because of traffic intensity on major roads. The A*-based approach adapted more effectively by selecting routes with lower estimated travel cost across the weighted graph.

Moreover, another set of experiment was done focusing more on positioning strategies: Static vs dynamic based on hill climbing. Findings indicate that in the static configuration, ambulances returned to predefined standby depots after completing a mission. While in the dynamic approach, ambulances were relocated toward positions expected to improve future city coverage and reduce travel distance to high-demand areas.

The dynamic positioning strategy produced more stable coverage during simulations involving multiple simultaneous emergencies. Queue growth was reduced and ambulances remained more evenly distributed across the environment compared to the static baseline.

<i>Metric</i>	<i>Static Positioning</i>	<i>Dynamic Positioning</i>
<i>Average Response Time</i>	<i>12.34 min</i>	<i>9.15 min</i>
<i>Emergency Coverage</i>	<i>62.5%</i>	<i>75.0%</i>
<i>Queue Growth</i>	<i>High</i>	<i>Moderate</i>

Table 9. Surge Scenario Evaluation

The experiments suggest that repositioning idle ambulances dynamically can improve ambulance availability during periods of high demand. Instead of concentrating ambulances around fixed standby depots, the optimization process redistributed resources toward more strategically useful areas within the simulated city.

The Hill Climbing optimization process itself was also analyzed during the evaluation phase. In most experiments, the algorithm converged relatively quickly and produced stable repositioning behavior. Random restarts helped reduce the probability of remaining trapped in poor local solutions and improved consistency between simulation runs.

Table 4 summarizes the main optimization observations.

<i>Metric</i>	<i>Observed Value</i>
<i>Average Iterations to Convergence</i>	<i>47</i>
<i>Average Optimization Time</i>	<i>0.87 s</i>
<i>Random Restarts Applied</i>	<i>Yes</i>
<i>Stable Convergence Behavior</i>	<i>Observed</i>

Table 10. Hill Climbing Optimization Performance

Several visualizations were generated to analyse system behavior and routing efficiency. Response-time histograms were used to compare dispatch strategies, while traffic heatmaps highlighted congestion intensity across different road segments. Ambulance movement visualizations also showed how routes changed dynamically according to traffic conditions. In addition, Hill Climbing convergence plots were generated to observe optimization progress over successive iterations.

Overall, the experiments indicate that combining traffic-aware routing with dynamic ambulance repositioning can improve emergency response performance within simulated urban environments. Although the evaluation was conducted using a simplified simulation framework, the obtained results consistently showed advantages over static positioning and purely distance-based dispatch strategies.

Discussion

The simulation results suggest that considering road congestion during ambulance assignment can improve overall response efficiency. In many test cases, the routing strategy based on A* produced shorter travel times than the simple distance-based method because it selected routes according to estimated movement cost on the road network rather than direct proximity alone.

Some limitations still remain in the current version of the system. Emergency events were generated using a Poisson model to simplify the simulation process, but real emergency activity is usually less regular and depends on several external factors such as traffic density, weather conditions, population movement, and time of day. Another fact to note is that we are assuming in this simulation instant dispatch communication, which is rarely the case in actual emergencies and operations.

The other limitation is the size and realism of the environment itself. The graph we implemented was suitable for testing and comparing algorithms but real metropolitan networks are much larger and more complex with additional operational constraints. Factors such as hospital admission delays, severe weather disruptions, and coordination between different emergency services were not fully represented in the current implementation.

In future works, several paths could be taken to improve this system especially with other optimization techniques such as Genetic Algorithms and RL (Reinforcement Learning), and that is to improve ambulance allocation and repositioning. Moreover, predictive methods and models can be utilized to flag some areas with a higher risk or likelihood of emergencies therefore allowing to adjust ambulance distribution.

In summary, through this project we demonstrated how basic search algorithms and optimization methods can be integrated to simulate an environment allowing us to study furthermore dispatch strategies.

Conclusion

This project presented a simulation based ambulance dispatch and routing system designed to improve emergency response efficiency in dynamic urban environments. By combining Real Time A* routing with Hill Climbing ambulance repositioning, the system was able to adapt to changing traffic conditions and emergency demand throughout the simulation.

The experimental results showed that traffic aware A* dispatch generally performed better than simple distance-based dispatch methods, especially during periods of heavy congestion. Dynamic ambulance repositioning also improved emergency coverage and reduced response delays during high demand scenarios compared to static standby positioning.

In addition to the implemented algorithms the project provided a complete simulation environment that combined graph based routing, traffic modeling, stochastic emergency generation and realtime ambulance movement. This allowed different strategies to be tested and compared under realistic urban conditions.

Although the system remains a simplified simulation of real emergency services the project demonstrates how AI searches and optimization techniques can support smarter and more efficient ambulance dispatch systems.

overall, the work highlights the importance of intelligent routing and adaptive resource management in improving emergency response performance and provides a solid foundation for future improvements and research.

References

Brotcorne, L., Laporte, G., & Semet, F. (2003). Ambulance Location and Relocation Models. *European Journal of Operational Research*, 451-463.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 100-107.

Ichoua, S., Gendreau, M., & Potvin, J.-Y. (2000). Diversion Issues in Real-Time Vehicle Dispatching. *Transportation Science*, 426-438.

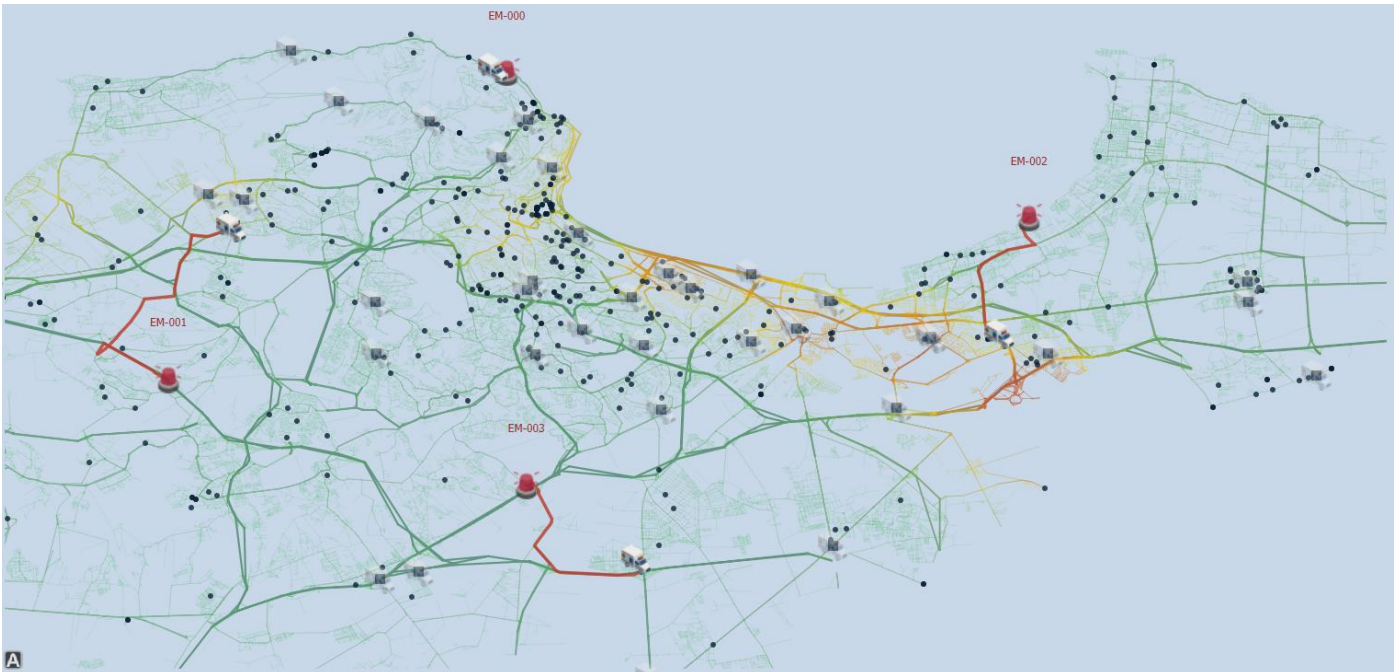
Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson.

Appendix A:

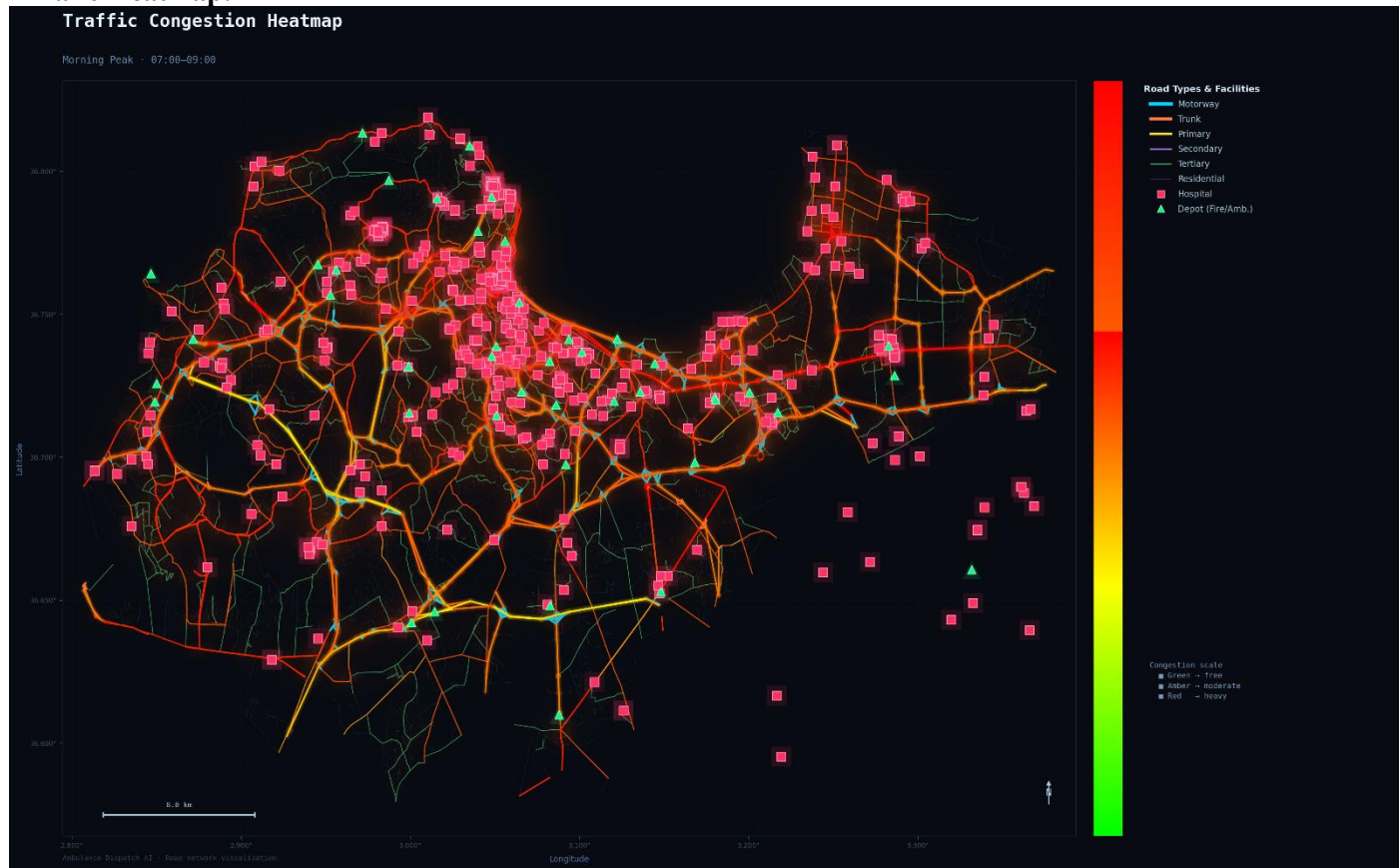
Road Network Visualisation:



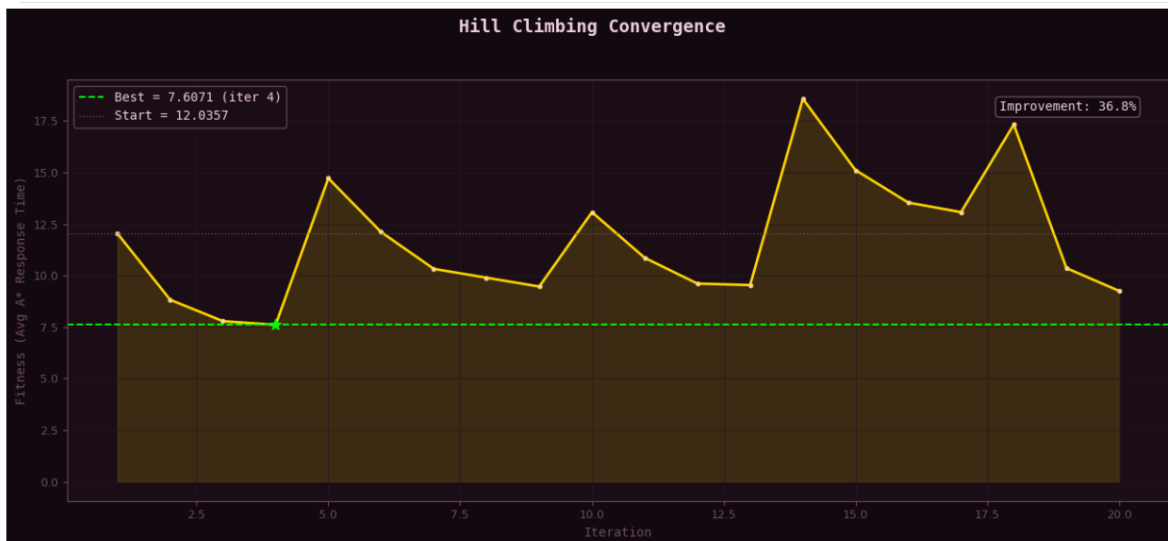
Ambulance Dispatch example:



Trrafic Heatmap:



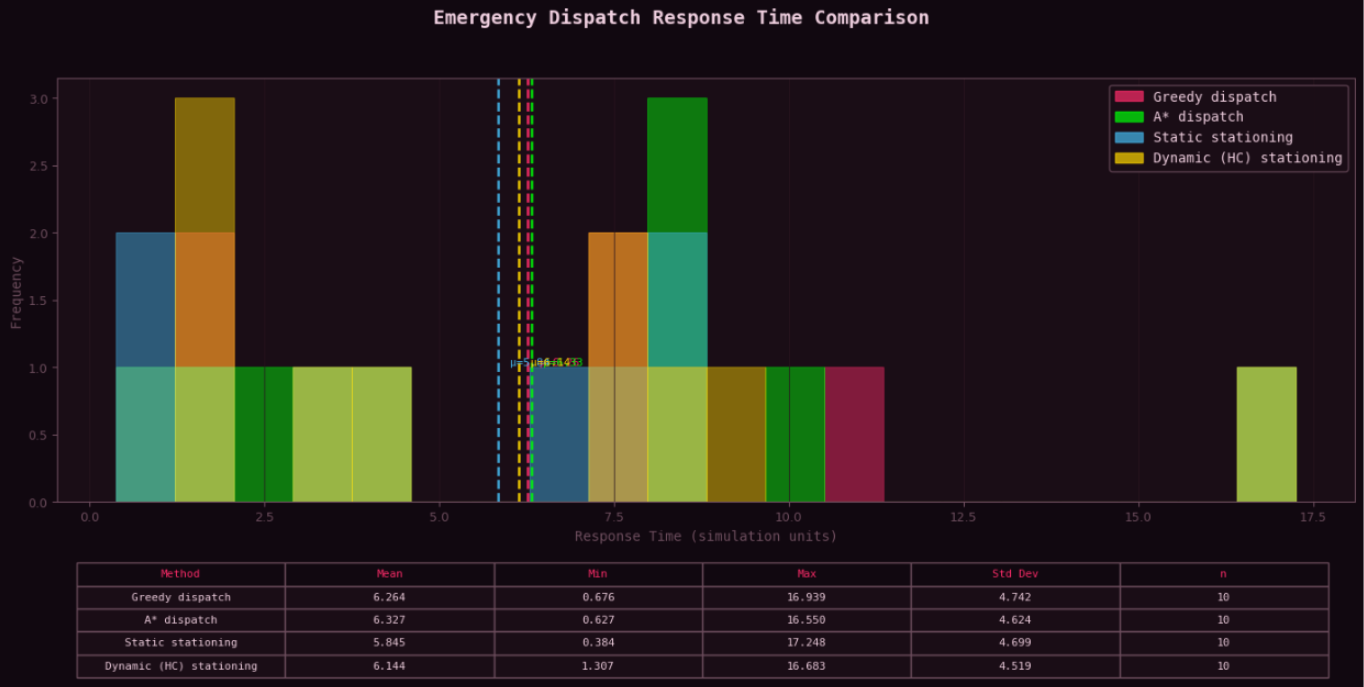
Hill Climbing Convergence:



```

greedy    count=10    mean=6.26
astar     count=10    mean=6.33
static    count=10    mean=5.84
dynamic   count=10    mean=6.14

```

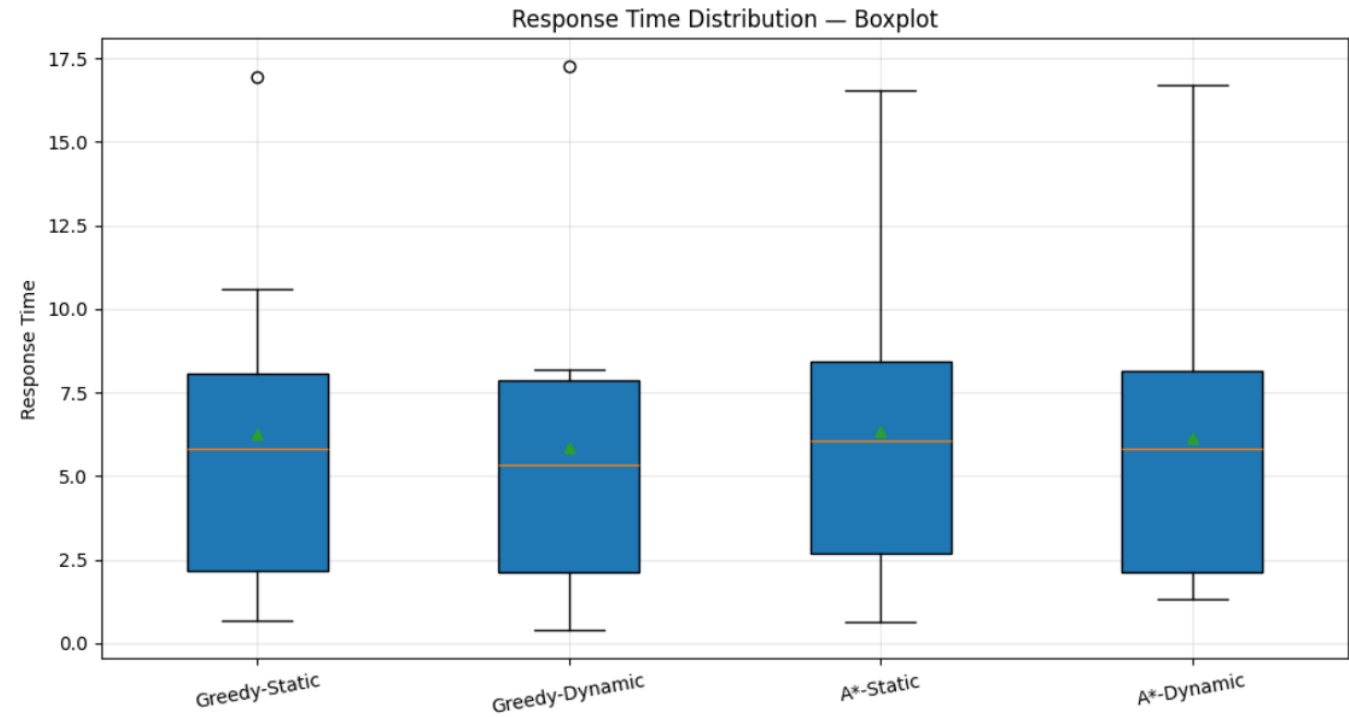


- **Box Plot:**

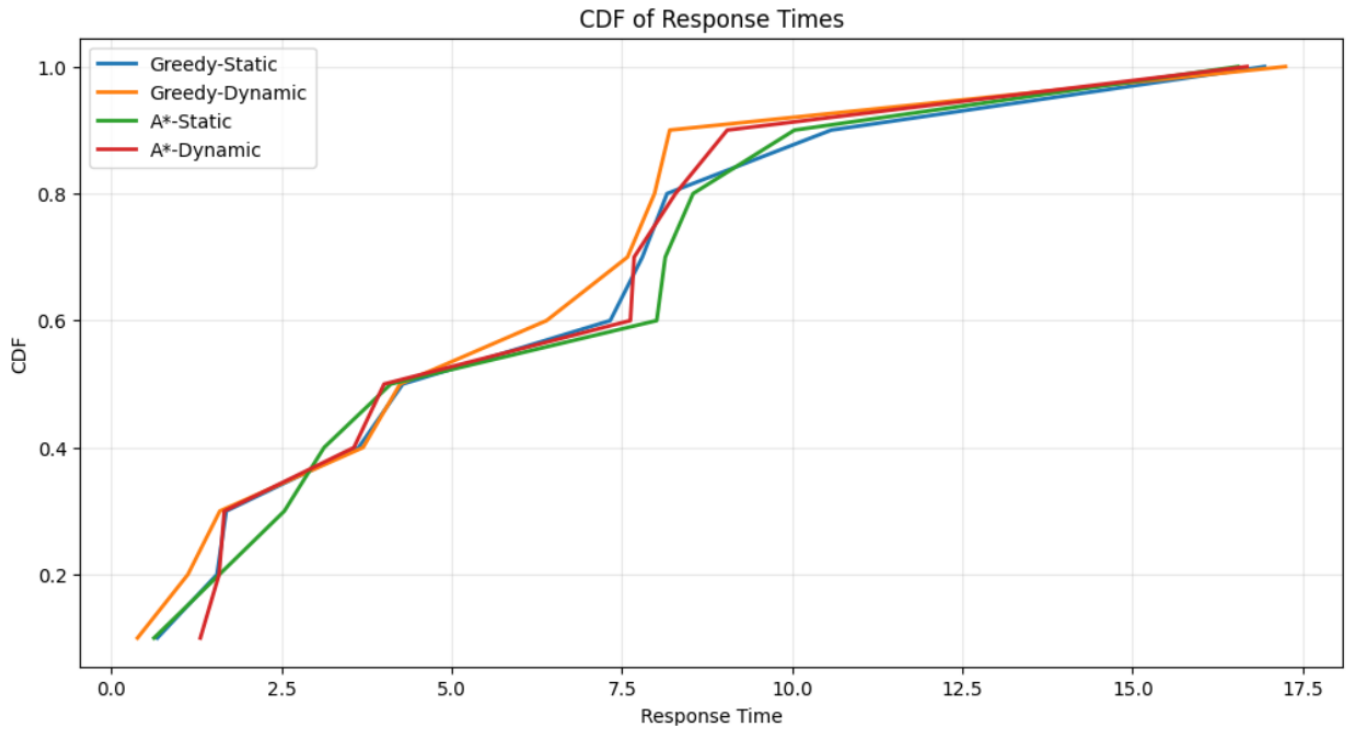
```

Greedy-Static    count=10    mean=6.26
Greedy-Dynamic   count=10    mean=5.84
A*-Static        count=10    mean=6.33
A*-Dynamic       count=10    mean=6.14

```

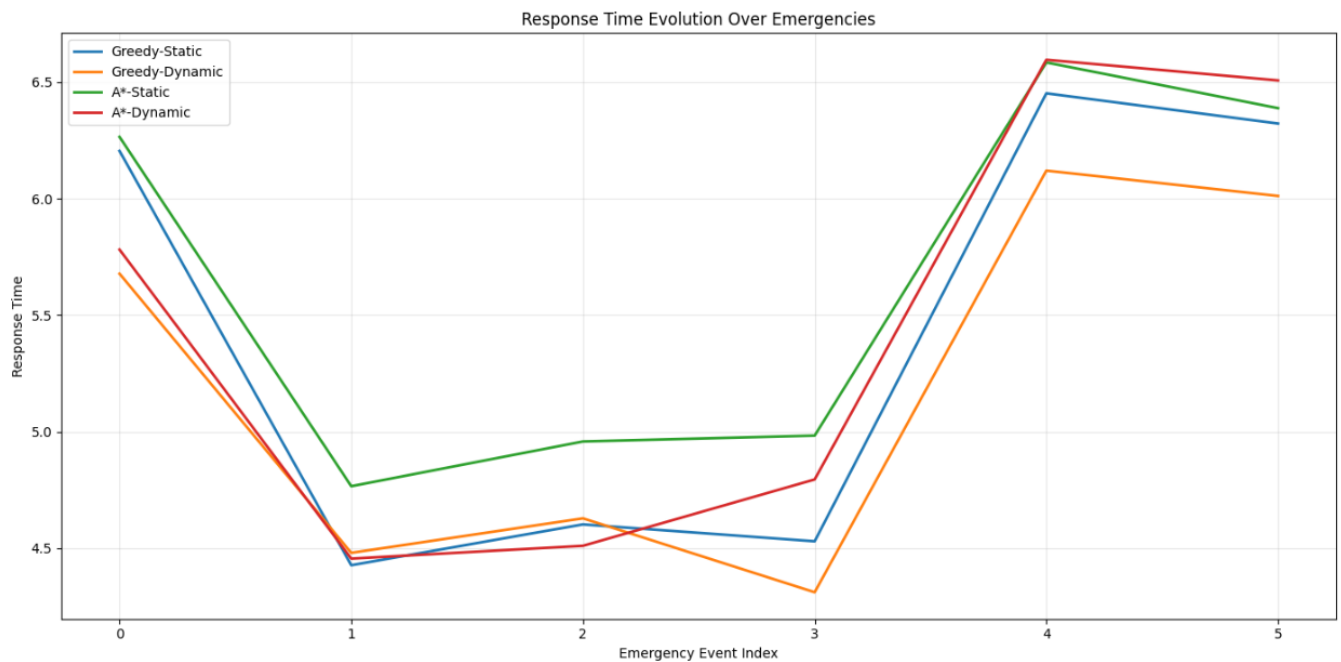


- **CDF of response Time:**



- Response time evolution over emergencies:

Greedy-Static	count=10	avg=6.26
Greedy-Dynamic	count=10	avg=5.84
A*-Static	count=10	avg=6.33
A*-Dynamic	count=10	avg=6.14



Example outputs from the terminal:

- Nodes init:

```
✓ 43 ambulances initialised
Ambulance(id=0, state=IDLE, node={'id': 'd0', 'node_id': 691187211, 'name': 'Unité de Protection civile الحماية المدنية', 'lat': 36.809002, 'lon': 3.034903, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=1, state=IDLE, node={'id': 'd1', 'node_id': 291997244, 'name': 'Unité de Protection civile', 'lat': 36.791047, 'lon': 3.048186, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=2, state=IDLE, node={'id': 'd2', 'node_id': 1786629895, 'name': 'Unité de Protection civile', 'lat': 36.813482, 'lon': 2.971606, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=3, state=IDLE, node={'id': 'd3', 'node_id': 958556549, 'name': 'Protection Civile', 'lat': 36.738891, 'lon': 3.050627, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=4, state=IDLE, node={'id': 'd4', 'node_id': 529670604, 'name': 'Unité de protection civile Birtouta', 'lat': 36.642387, 'lon': 3.000564, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=5, state=IDLE, node={'id': 'd5', 'node_id': 11563583801, 'name': 'Pompiers Cheraga', 'lat': 36.756932, 'lon': 2.952625, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=6, state=IDLE, node={'id': 'd6', 'node_id': 334258573, 'name': 'Protection civile Bab Ezzouar', 'lat': 36.721033, 'lon': 3.179947, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=7, state=IDLE, node={'id': 'd7', 'node_id': 326511227, 'name': 'Unité de Protection civile', 'lat': 36.722916, 'lon': 3.065685, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=8, state=IDLE, node={'id': 'd8', 'node_id': 2535738306, 'name': 'Unité de Protection civile', 'lat': 36.610085, 'lon': 3.087768, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=9, state=IDLE, node={'id': 'd9', 'node_id': 319882830, 'name': 'Unité de Protection Civile', 'lat': 36.754232, 'lon': 3.064423, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=10, state=IDLE, node={'id': 'd10', 'node_id': 2355999261, 'name': 'Unité de Protection civile', 'lat': 36.720216, 'lon': 3.180241, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=11, state=IDLE, node={'id': 'd11', 'node_id': 3015239414, 'name': 'Protection civile Sidi Fredj', 'lat': 36.764235, 'lon': 2.846692, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=12, state=IDLE, node={'id': 'd12', 'node_id': 3901994118, 'name': 'Unité de Protection civile', 'lat': 36.741338, 'lon': 2.87158, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=13, state=IDLE, node={'id': 'd13', 'node_id': 26616005, 'name': 'Protection civile', 'lat': 36.741549, 'lon': 3.12231, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=14, state=IDLE, node={'id': 'd14', 'node_id': 2259163239, 'name': 'Unité de Protection civile', 'lat': 36.73361, 'lon': 3.082393, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=15, state=IDLE, node={'id': 'd15', 'node_id': 2354933975, 'name': 'Unité Nationale d'Instruction et d'Intervention de Dar El Beidha', 'lat': 36.722652, 'lon': 3.200436, 'ambulance_count': 3, 'source': 'osm'})
Ambulance(id=16, state=IDLE, node={'id': 'd16', 'node_id': 501931743, 'name': 'Unité secondaire de Rouiba', 'lat': 36.739003, 'lon': 3.282657, 'ambulance_count': 3, 'source': 'osm'})
```

- Emergency generating:

In [6]:

```
SIM_DURATION = 300.0
LAMBDA_RATE = 0.08

# — compute bounds correctly —
xs = [n.lon for n in graph.nodes.values()]
ys = [n.lat for n in graph.nodes.values()]

generator = PoissonEmergencyGenerator(
    lambda_rate=LAMBDA_RATE,
    max_x=max(xs),
    max_y=max(ys),
    min_x=min(xs),
    min_y=min(ys),
    integer_coords=False
)

# — generate events —
all_events = []
current_time = 0.0

while current_time < SIM_DURATION:
    event = generator.generate_next_arrival(current_time)
    all_events.append(event)
    current_time = event.timestamp

print(f"✓ {len(all_events)} emergency events generated")
print(f"λ = {LAMBDA_RATE}/min | duration = {SIM_DURATION} min")
print(f"First 3 events: {all_events[:3]}")
```

```
✓ 28 emergency events generated
λ = 0.08/min | duration = 300.0 min
First 3 events: [Emergency(id=1, x=3.03223256745629, y=36.586424968308414, time=12.20, assigned=False), Emergency(id=2, x=3.304336093797913, y=36.70120623264541, time=26.67, assigned=False), Emergency(id=3, x=3.2573880391485286, y=36.76299533426498, time=37.37, assigned=False)]
```

- A* star stats:

A* + REAL-TIME A* UNIT TESTS

- ✓ A* path found | cost = 5.33 min
- ✓ A* self-test passed
- ✓ RT-A* initial path found | cost = 51.00
- ✓ Blocking edge and rerouting...
- ✓ RT-A* reroute success | cost = inf
- ✓ ALL A* + RT-A* TESTS PASSED

- Hill climbing stats:

HILL CLIMBING – STANDBY OPTIMISATION

- ✓ HillClimbing compatibility patch applied
 - ✓ Emergency nodes mapped : 28
 - ✓ Fast fitness approximation enabled
 - ✓ Optimisation ambulances : 5
 - ✓ Emergency samples : 8
 - Restart 1: Fitness = 9.88
 - Restart 2: Fitness = 11.38
 - ✓ Hill Climbing complete
- Best standby positions:
- Ambulance 00 → Node 816346387
 - Ambulance 01 → Node 10022721607
 - Ambulance 02 → Node 1180700369
 - Ambulance 03 → Node 3025646951
 - Ambulance 04 → Node 2559997006
- ✓ Best fitness score : 9.88
 - ✓ Approx response score : 9.88
 - ✓ Convergence plot not available (method missing in class)

- SM with HC:

- ✓ StandbyManager initialised with HillClimbing
- ✓ HC ready for dynamic recomputation
- ✓ HC positions:
[816346387, 10022721607, 1180700369, 3025646951, 2559997006]
- Restart 1: Fitness = 7.61
- Restart 2: Fitness = 9.46
- Restart 3: Fitness = 9.54
- Restart 4: Fitness = 13.07
- Restart 5: Fitness = 9.25
- ✓ recomputed positions:
[3678591394, 330410672, 956484251, 317140812, 6332915610]

- Initial comparison:

FINAL RESULTS

Greedy-Static : avg=6.26 min | events=10
Greedy-Dynamic : avg=5.84 min | events=10
A*-Static : avg=6.33 min | events=10
A*-Dynamic : avg=6.14 min | events=10

🏆 BEST CONFIGURATION: Greedy-Dynamic

✓ Average response time: 5.84 min

- Comparing with full configuration:

Config	N	Mean	Median	Min	Max	P95	Std
Greedy-Static	10	6.26	5.80	0.68	16.94	10.57	5.00
Greedy-Dynamic	10	5.84	5.32	0.38	17.25	8.20	4.95
A*-Static	10	6.33	6.06	0.63	16.55	10.04	4.87
A*-Dynamic	10	6.14	5.82	1.31	16.68	9.05	4.76

🏆 BEST CONFIGURATION: Greedy-Dynamic

✓ Mean response time : 5.84 min

FULL STATISTICAL SUMMARY

Events generated : 20
 λ : 0.08 /min
Duration : 300.0 min
Ambulances : 43
Hospitals : 453

[Greedy-Static]

Assigned : 10
Mean : 6.22 min
Median : 2.82 min
Std Dev : 7.23 min
95th pctlile : 19.41 min

[Greedy-Dynamic]

Assigned : 10
Mean : 6.91 min
Median : 4.28 min
Std Dev : 6.89 min
95th pctlile : 19.41 min

[A*-Static]

Assigned : 10
Mean : 6.63 min
Median : 2.95 min
Std Dev : 7.81 min
95th pctlile : 19.67 min

[A*-Dynamic]

Assigned : 10
Mean : 6.37 min
Median : 2.76 min
Std Dev : 7.31 min
95th pctlile : 19.74 min

HC best positions : [346899801, 13167563550, 10908328933, 334269356, 3585290838]

HC avg response time : -5.25 min

Surge (7 simultaneous) : 7 assigned, 0 unassigned, 0 capacity violations

DETAILED RESPONSE TIME ANALYSIS

=====

RESPONSE TIME ANALYSIS

=====

Metric	Greedy	A*
Average	6.22	6.37
Minimum	0.63	1.14
Maximum	19.81	20.02
Std Dev	7.23	7.31
# Emergencies	10	10

→ Greedy is 2.5 % faster than A* on this run

=====

Appendix B: Detailed work plan and tasks

	Pair A	Pair B	Pair C
Members	Allali Bouchra + Hamadach Thawriyya	Immessaoudene Malak Ikram + Moumani Malek Nourhene	Bentata Ikram Amina + Ferhaoui Amina
Algorithm owned	Real-Time-based A* pathfinding with dynamic traffic weight updates	Hill Climbing for optimal ambulance standby locations	Greedy dispatch vs A*-assisted dispatch + simulation loop
Also responsible for	Map animation, A* vs Greedy experiment + final notebook cleanup, dashboard, App Design	Road graph + traffic model, traffic heatmap, static vs dynamic experiment, Dataset Building + Results + Discussion report sections.	Emergency generator, surge scenario + histograms, Problem Solving Conclusion report sections, demo prep, Final report cleanup

Leader's prep work:

Apr 10-17	- Setting up shared repo, Jupyter notebook template, and Data/ folder structure. - Setting up graph data formatting with M3 before any coding begins. - Research for articles to be used in the project and methodology
------------------	---

M1 - Allali Bouchra

Apr 10-17	- Data managing and fetching, including on-site visits to various hospitals across the city of Algiers. - Building the map.json file to match the assigned profiles - Fetched the data using the assigned script
Apr 18-24	- Implement standard A* on the road graph with a Harver sign heuristic. - Write unit tests: verify shortest path on a known small graph. - Fixed The node.py and graph logic
Apr 25-May 1	- Plug A* into the dispatch system for Goal 1 (ambulance to scene) and Goal 2 (scene to hospital). - Attend and run the Week 3 team integration sync.
May 2-8	- Animate ambulance movement on the map. - Highlight high-weight (congested) edges in red. - Make sure path re-routing is visible when an edge is blocked.
May 9-16	- Building The full Animated map, including the implemented app to fully incorporate the algorithms and display the dispatch logic. - Designing the interface of the optimized app and correcting issues regarding M6's work of the dashboard. - Place hospitals and depots at nodes on the graph. - Responsible for the Application design part of the report

M2 - Hamadach Thawriyya	
Apr 10-17	- Reaearch and regulated a summary paper regarding A* search (at that time it was ambiguous to u sstudents)
Apr 18-24	- Extend A* to Real-Time-based A* (inspired by LRTA* and D*) - Test the case where an edge becomes blocked after the ambulance has already started moving.
Apr 25-May 1	- Build A*-based time dispatch: instead of picking the geographically closest ambulance, pick the one with the shortest A* travel time. - Compare ETA results to Greedy.
May 2-8	- Run the A* dispatch vs Greedy dispatch experiment over 100+ simulated emergencies. - Record average, minimum, and maximum response time for both methods. - Produce a comparison table.
May 9-16	- Writing the notebook and final cleanup: add explanatory text cells, docstrings, ensure all file names are meaningful, putting cells to test all files and algos - Run the notebook end-to-end with zero errors. - Fixing Issues and overriding some functions on other files for final integration - Responsible for the discussion part of the report

M3 - Immessaoudene Malak Ikram	
Apr 10-17	- Build the road graph: create node objects (intersections) and edge objects (road segments) with base distance weights. - Export the graph as CSV or JSON for the whole team to use.
Apr 18-24	- Implement Hill Climbing core algorithm. - State = the standby position of each idle ambulance (one node per ambulance). - Fitness function = average A* response time computed over all historical accident coordinates.
Apr 25-May 1	- Connect Hill Climbing output to idle ambulance positioning. - After each hospital drop-off, move the ambulance to the node that HC identified as the best standby position.
May 2-8	- Build the traffic heatmap: colour road edges by their current congestion weight using matplotlib or plotly. - Overlay emergency event locations as scatter points on the same map.
May 9-16	- Write the Dataset Building section: document graph stats, Poisson parameters, summary statistics, and any preprocessing. - Write the Results and Analysis section with all comparison tables and embedded figures.

M4 - Moumani Malek Nourhene	
Apr 10-17	- Add the time-of-day traffic model to the graph: apply x2-3 weight multipliers on main roads during rush hour, and x0.5 at night.
Apr 18-24	- Add random restarts to the Hill Climbing algorithm so it can escape local optima. - Log the fitness score after every iteration. Run tests with 3, 5, and 10 ambulances.
Apr 25-May 1	- Implement the static stationing baseline: after each job, ambulances return to their designated Home Depot.

	- Ensure both static and dynamic modes are run on the identical emergency sequence for a fair comparison.
May 2-8	- Run the static vs dynamic stationing experiment. - Run the surge scenario test from M6's work - Record and document all metrics that will go into the Results section.
May 9-16	- Write the Discussion section: interpret the results, acknowledge limitations, and suggest future work.

M5 - Bentata Ikram Amina

Apr 10-17	- Write the Ambulance class and Hospital class. - Implement the ambulance state machine with states: idle, dispatched, at_scene, to_hospital, idle. - Each state transition must update position and availability.
Apr 18-24	- Implement Greedy dispatch: when an emergency arrives, assign the available ambulance with the smallest Euclidean distance to the emergency coordinates.
Apr 25-May 1	- Run the full simulation end-to-end combining A* routing, Hill Climbing standby positioning, and dispatch logic. - Identify and fix integration bugs between modules and imports
May 2-8	- Build the response time histogram with all methods overlaid (Greedy, A*-dispatch, static stationing, dynamic stationing) on the same chart. - Build the Hill Climbing convergence plot (fitness score vs iteration number).
May 9-16	- Write the Problem-Solving Techniques section.

M6 - Ferhaoui Amina (Team Leader)

Apr 10-17	- Prep (First part) - Built the main simulation loop using discrete time steps and an event queue. - Implement the Poisson emergency generator.
Apr 18-24	- Setting up the comparison framework: run the same fixed sequence of emergencies with Greedy dispatch and with A*-based dispatch separately. - Logged the response time for every emergency under both methods. - Implemented the simulation_engine and optimized Standby Manager for hill climbing
Apr 25-May 1	- Implement the surge scenario - Verified that the capacity constraint (each ambulance carries exactly 1 patient) is correctly enforced and no patient is double-assigned.
May 2-8	- Built the simulation backbone - Built the Dashboard but failed miserably
May 9-16	- Leading demo preparation. - Write the demo script covering all three algorithms. - Organize a mock demo with all six members. - Write Abstract + Introduction + State of the Art - Coordinated the final submission - Verified all APA references are consistent. - Refined and re-edited Data set building, Discussion and analysis parts of the report
Recurrent	- Git management - Resolving Format conflicts - Handling errors and fixes of integration alongside my pair

